

Reporte: Sonus

Luana Sofia Alvarez Molina

8 de mayo de 2026

Resumen

Desarrollo de un sistema de gestión musical para almacenar y consultar archivos MP3 mediante una base de datos. Incluye minería de metadatos, interfaz gráfica y persistencia de datos, aplicando diseño modular y patrones como DAO, Singleton y MVC.

1. Introducción

El objetivo de este proyecto es desarrollar un sistema de gestión de música que permita almacenar, consultar y manipular información de archivos MP3 mediante una base de datos.

El sistema está compuesto por varios módulos principales: una base de datos musical, un minero de archivos capaz de extraer metadatos en formato ID3v2.4, una interfaz gráfica para la interacción del usuario, y un componente de consulta que permite realizar búsquedas lógicas complejas sin necesidad de escribir consultas SQL directamente.

Este proyecto busca integrar conceptos de bases de datos, procesamiento de archivos y diseño de interfaces gráficas en una sola aplicación funcional.

2. Análisis

2.1. Objetivos

- Desarrollar un sistema capaz de almacenar información de archivos MP3 en una base de datos.
- Implementar un minero que extraiga metadatos en formato ID3v2.4.
- Diseñar una interfaz gráfica para visualizar y modificar la información.
- Permitir consultas a la base de datos sin necesidad de SQL.
- Aplicar pruebas unitarias para validar los módulos del sistema.
- (Opcional) Implementar un reproductor de musica que reproduzca los archivos MP3.

3. Diseño

El sistema fue dividido en los siguientes módulos principales: el módulo de base de datos, el minero de archivos, la interfaz gráfica y el módulo de consultas.

3.1. Arquitectura

El sistema fue diseñado siguiendo una arquitectura modular, separando las distintas responsabilidades en componentes independientes. Esta separación permite mejorar la mantenibilidad, escalabilidad y claridad del código.

Los módulos del sistema son:

- **Domain:** Contiene las entidades principales del sistema, como canciones, álbumes y artistas, así como las subdivisiones de los artistas en grupos y personas, junto con su relación. Es una abstracción en objetos de las tablas de base de datos.
- **DB/DAO:** Encargado del acceso a la base de datos y la persistencia de la información.
- **Miner:** Responsable de analizar archivos MP3 y extraer metadatos.
- **GUI:** Proporciona el controlador de la interfaz gráfica para la interacción con el usuario.
- **Query:** Diseñado para permitir consultas sin necesidad de SQL (no implementado).
- **Resources/ui:** Maquetado de la interfaz gráfica, es decir, la vista.

3.2. Patrones de diseño

Durante el desarrollo del sistema se emplearon varios patrones de diseño de manera implícita, con el objetivo de mejorar la organización, mantenibilidad y claridad del código.

El módulo de acceso a la base de datos, ubicado en el directorio `db/dao`, sigue el patrón *Data Access Object (DAO)*, separando la lógica de persistencia de la lógica del modelo. Cada entidad del dominio cuenta con sus propios métodos de acceso, si bien es un solo objeto DAO se dividió en clases parciales (por ejemplo, `songDAO`, `albumDAO`, `personDAO`) para facilitar la legibilidad, desarrollo de pruebas unitarias y mantenimiento del mismo, lo que permite encapsular las operaciones sobre la base de datos y evitar SQL en el resto del código.

Asimismo, se utilizó el patrón *Singleton* en la clase `DatabaseManager`, con el fin de garantizar que exista una única instancia encargada de manejar la conexión a la base de datos durante la ejecución del programa. Esto permite centralizar el acceso a la base de datos, evitar múltiples conexiones simultáneas y facilitar el control de apertura y cierre de la misma.

Finalmente, la organización general del sistema sigue el patrón *Modelo-Vista-Controlador (MVC)*. Los módulos `db`, `domain`, y `miner` en `src/` representan el modelo, ya que contienen las entidades principales del sistema así como la lógica del programa; el módulo `src/gui` actúa como el controlador; y el módulo `resources/ui` actúa como la vista.

Esta separación de responsabilidades permite desacoplar los distintos componentes del sistema, facilitando su mantenimiento y extensión.

3.3. Diagrama de clases

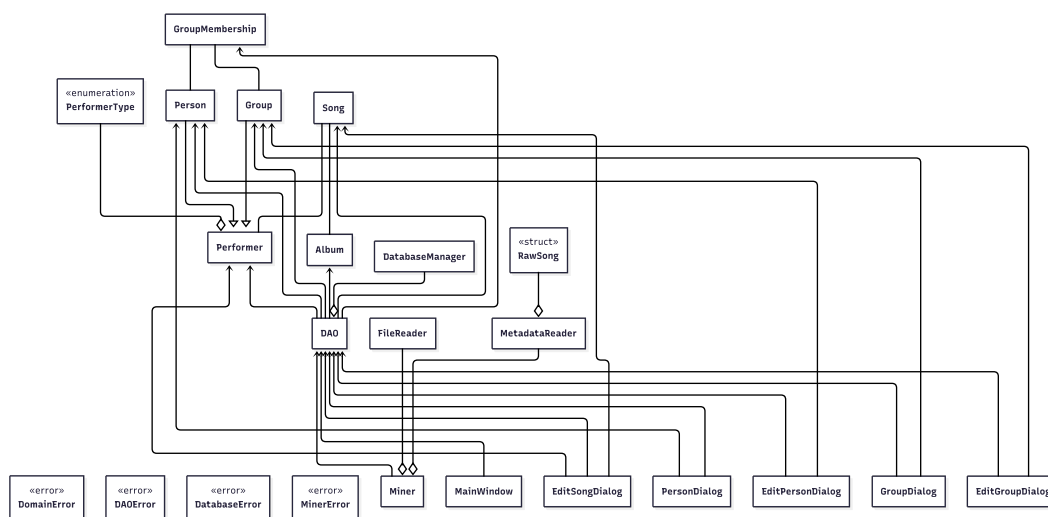


Figura 1: Diagrama de clases simplificado.

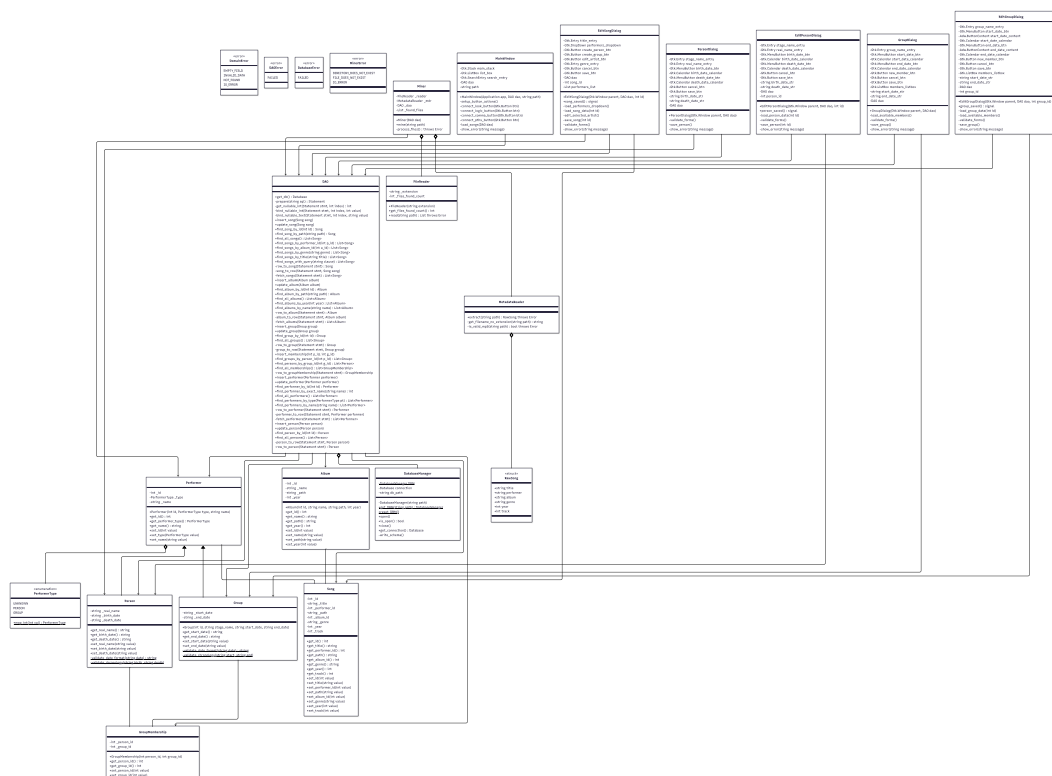


Figura 2: Diagrama de clases completo.

[Haga clic aquí para extraer el diagrama completo \(PDF\)](#)

4. Implementación

El sistema fue implementado utilizando el paradigma de programación orientada a objetos, empleando el lenguaje Vala debido a su integración con bibliotecas de bajo nivel en C (como `taglib_c` para el manejo de metadatos) y su facilidad para el desarrollo de interfaces gráficas mediante GTK y Blueprints.

La implementación se realizó siguiendo la arquitectura modular descrita previamente, dividiendo el sistema en componentes independientes como el módulo de dominio, acceso a datos (DAO), minería de archivos y la interfaz gráfica. Esta organización permitió separar responsabilidades y facilitar el mantenimiento del código.

Asimismo, se aplicaron distintos patrones de diseño, como DAO, Singleton y MVC; y se adoptó un enfoque guiado por pruebas para validar el correcto funcionamiento de los distintos módulos durante el desarrollo.

Adicionalmente, se optó por utilizar nombres de clases, métodos y variables en inglés, con el fin de seguir convenciones ampliamente adoptadas en el desarrollo de software y mantener coherencia con las bibliotecas y métodos propios de Vala, los cuales están definidos en este idioma. Esto permite evitar ambigüedades y facilita la comprensión del código.

De la misma manera, se adoptó el uso de la convención `snake_case` en nombres de funciones y variables, en concordancia con el estilo utilizado en las bibliotecas subyacentes en C sobre las que Vala se construye, lo que contribuye a una mayor consistencia y legibilidad del código.

4.1. Modulo de dominio

El módulo `domain` se encarga de abstraer las tablas de la base de datos como objetos, usando clases como `song`, `album` y `performer` para manipular esta información a lo largo del programa.

Estas clases no solo representan los datos, sino que también encapsulan la lógica y las restricciones asociadas a cada entidad, validando que los objetos se mantengan en un estado consistente. Por ejemplo, se verifican condiciones como campos obligatorios no vacíos, identificadores válidos y formatos correctos en atributos como rutas de archivos.

Además, se implementan métodos *getter* y *setter* que controlan el acceso y modificación de los datos, asegurando que cualquier cambio respete las invariantes del sistema. De esta forma, el módulo de dominio actúa como una capa intermedia entre la base de datos y el resto de la aplicación, centralizando la lógica del modelo.

4.2. Modulo de base de datos y DAO

El acceso a la base de datos se implementó utilizando SQLite3. Se desarrolló una capa de acceso a datos mediante el objeto DAO, encargado de encapsular las operaciones de consulta y manipulación de la información, evitando el uso directo de SQL en el resto de la aplicación.

El DAO centraliza todas las operaciones sobre la base de datos, proporcionando métodos para insertar, actualizar y consultar entidades del dominio como canciones, álbumes y artistas. Para facilitar su organización, mantenimiento y desarrollo de pruebas se dividió en clases parciales (por ejemplo, `songDAO`, `albumDAO`, `personDAO`), permitiendo separar la lógica correspondiente a cada entidad.

Asimismo, se implementaron métodos auxiliares como `row_to_object()` y `object_to_row()` para ayudarnos al momento de realizar las consultas. Además de otros métodos auxiliares como `prepare()` para preparar las consultas SQL o `bind_nullable_int()` y `bind_nullable_text()` para insertar el valor en la consulta si existe o insertar NULL si no.

Además, se implementó una clase `DatabaseManager` que sigue el patrón Singleton, encargada de gestionar la

conexión a la base de datos, garantizando que exista una única instancia durante la ejecución del programa y centralizando el control de apertura y cierre de la misma.

4.2.1. Ejemplo de implementación del DAO: Inserción

A continuación se muestra un fragmento para insertar una canción y el metodo auxiliar que nos ayuda a escribir los valores del objeto `cancion` que recibe `insert_song()` en los valores de la consulta:

```
public void insert_song(Song song) throws Error {
    var existing = this.find_song_by_path(song.get_path());
    if (existing != null)
        return;

    var sql = "INSERT INTO rolas (title,
                                id_performer,
                                id_album,
                                path,
                                track,
                                year,
                                genre)
              VALUES (?, ?, ?, ?, ?, ?, ?)";
    var stmt = prepare(sql);
    song_to_row(stmt, song);
    stmt.step();
}

private void song_to_row(Statement stmt, Song song) throws Error {
    stmt.bind_text(1, song.get_title());
    bind_nullable_int(stmt, 2, song.get_performer_id());
    bind_nullable_int(stmt, 3, song.get_album_id());
    stmt.bind_text(4, song.get_path());
    bind_nullable_int(stmt, 5, song.get_track());
    bind_nullable_int(stmt, 6, song.get_year());
    bind_nullable_text(stmt, 7, song.get_genre());
}
```

El método `insert_song()` recibe una canción y verifica si ya existe en la base de datos. En caso de que exista, no se realiza la inserción. En caso contrario, se prepara la consulta de inserción en SQL y, haciendo uso de `song_to_row()`, se insertan los datos correspondientes en la consulta preparada. Si algún campo de la canción es nulo, se inserta NULL; de lo contrario, se inserta el valor correspondiente.

4.2.2. Ejemplo de implementación del DAO: Obtención

A continuación se muestra un fragmento para obtener una canción y el metodo auxiliar que nos ayuda a convertir los datos que nos regresa SQL en forma de tabla en un objeto de tipo `song`:

```
public Song? find_song_by_id(int id) throws Error {
    var stmt = prepare("SELECT * FROM rolas WHERE id_rola = ?");
    stmt.bind_int(1, id);
    if(stmt.step() == Sqlite.ROW)
        return row_to_song(stmt);
}
```

```

        return null;
    }

    private Song row_to_song(Statement stmt) throws Error {
        int? performer_id = get_nullable_int(stmt, 1);
        int? album_id = get_nullable_int(stmt, 2);
        int? track = get_nullable_int(stmt, 5);
        int? year = get_nullable_int(stmt, 6);

        var s = new Song(
            stmt.column_int(0),
            stmt.column_text(4),
            performer_id,
            stmt.column_text(3)
        );

        s.set_album_id(album_id);
        s.set_track(track);
        s.set_year(year);
        s.set_genre(stmt.column_text(7));

        return s;
    }
}

```

El método `find_song_by_id()` recibe un entero (id de la canción a buscar) y hace la consulta para obtener la canción con el id igual al entero dado. En caso de existir, se ayuda de `row_to_song()` para convertir los datos obtenidos en un objeto de tipo canción. En caso de no existir se regresa NULL.

4.2.3. Ejemplo del DataBaseManager

A continuación se muestran las variables de clase, el constructor y el método utilizado para obtener la instancia única del manejador de base de datos.

```

private static DatabaseManager? DBM = null;
private Database? connection = null;
private string db_path;

private DatabaseManager(string path) {
    this.db_path = path;
}

public static DatabaseManager get_DBM(string path = "data/sonus.db") {
    if (DBM == null)
        DBM = new DatabaseManager(path);
    return DBM;
}
}

```

Ahora bien, el siguiente es el método utilizado para abrir la conexión con la base de datos. Nos aseguramos de que el directorio y el archivo existan, si existen intentamos abrir la conexión, si algo falla lanzamos un error. En caso

de que no exista, al intentar abrir la conexión se crea el archivo y entonces escribimos el `schema.sql` sobre este.

```
public void open() throws Error {
    if (connection != null)
        return;

    File file = File.new_for_path(db_path);
    File parent = file.get_parent();
    if (parent != null && !parent.query_exists())
        parent.make_directory_with_parents();

    bool exists = file.query_exists();

    int rc = Sqlite.Database.open(db_path, out connection);
    if (rc != Sqlite.OK)
        throw new DatabaseError.FAILED("Error: Couldn't open the connection");

    if (!exists)
        write_schema();
}
```

4.3. Módulo de minería de datos

El módulo `miner` se encarga de recorrer el directorio del cuál queremos extraer la música, leer los metadatos, crear los objetos necesarios e insertarlos en la base de datos usando el DAO.

Para ello, se implementó la siguiente función en el archivo `src/miner/fileReader.vala` para recorrer recursivamente cada directorio y obtener cada archivo de música dentro de este:

```
public ArrayList<string> read(string path) throws Error{
    var file_list = new ArrayList<string>();
    var directory = File.new_for_path(path);

    if(!directory.query_exists())
        throw new MinerError.DIRECTORY_DOES_NOT_EXIST("The given directory doesn't exist.");

    var enumerator = directory.enumerate_children(FileAttribute.STANDARD_NAME + "," +
                                                FileAttribute.STANDARD_TYPE,
                                                FileQueryInfoFlags.NONE
                                                );

    FileInfo info;

    while((info = enumerator.next_file()) != null){
        var child = directory.get_child(info.get_name());
        var child_path = child.get_path();

        if(info.get_file_type() == FileType.DIRECTORY){
            try {
                file_list.add_all(read(child_path));
            } catch (Error e) {
                stderr.printf("[ERROR] Couldn't access to %s: %s\n", child_path, e.message);
            }
        }
    }
}
```

```

        } else if(info.get_file_type() == FileType.REGULAR){
            if(info.get_name().down().has_suffix(_extension)){
                file_list.add(child_path);
                _files_found_count++;
            }
        }
    }

    return file_list;
}

```

De igual manera se implemento la siguiente función en el archivo `src/miner/metadataReader.vala` para extraer todos los datos que deben venir en los metadatos de cada archivo MP3, y almacenarlos en un struct que creamos específicamente para esto y así facilitarnos el acceso a los datos al momento de querer insertarlos en la base de datos.

```

public struct RawSong{
    public string title;
    public string? performer;
    public string? album;
    public string? genre;
    public int? year;
    public int? track;
}

public RawSong? extract(string path) throws Error {
    if (!is_valid_mp3(path))
        return null;

    var file = new TagLib.File(path);
    if (file == null || !file.is_valid())
        return null;

    if (file.audioproperties == null || file.audioproperties.length == 0)
        return null;

    var raw = RawSong();
    var _title = file.tag.title;
    var _performer = file.tag.artist;
    var _album = file.tag.album;
    var _genre = file.tag.genre;
    var _year = (int)file.tag.year;
    var _track = (int)file.tag.track;

    raw.title      = (_title != null && _title.strip() != "") ?
                                                              _title : get_filename_no_extension(path);
    raw.performer  = (_performer != null && _performer.strip() != "") ? _performer : null;
    raw.album      = (_album != null && _album.strip() != "") ? _album : null;
    raw.genre      = (_genre != null && _genre.strip() != "") ? _genre : null;
    raw.year       = (_year > 0) ? (int?)_year : null;
    raw.track      = (_track > 0) ? (int?)_track : null;

    return raw;
}

```



```
}
```

Por último se muestra la función `process_files()`, utilizada dentro del método `mine()`, la cual se encarga de procesar cada uno de los archivos previamente encontrados en el directorio.

Esta función recorre la lista de rutas almacenadas en `_found_files` y, para cada archivo, primero verifica si la canción ya existe en la base de datos mediante `find_song_by_path()`. En caso de que exista, se omite su procesamiento para evitar duplicados.

Posteriormente, se extraen los metadatos del archivo utilizando `metadataReader`. Si no se logra obtener información válida, el archivo se descarta.

A partir de los metadatos obtenidos, se procesan los campos relacionados con el intérprete y el álbum. En el caso del intérprete, se limpia el nombre y se busca en la base de datos; si no existe, se inserta un nuevo registro y se obtiene su identificador. De manera similar, para el álbum se verifica si ya existe utilizando su ruta; en caso contrario, se crea e inserta uno nuevo.

Finalmente, se construye un objeto de tipo `Song` con la información recolectada y se inserta en la base de datos mediante el DAO. Una vez insertada, se recupera la canción para confirmar su registro y mostrar información de depuración.

En caso de que ocurra algún error durante el procesamiento de un archivo, este es capturado y reportado sin detener la ejecución del resto del proceso.

```
private void process_files() throws Error {
    foreach(string path in _found_files) {
        try {
            if(_dao.find_song_by_path(path) != null)
                continue;

            var raw = _mdr.extract(path);
            if(raw == null)
                continue;

            string album_path = Path.get_dirname(path);
            int? performer_id = null;
            int? album_id = null;

            if(raw.performer != null){
                string performer_name = raw.performer.strip().replace("\u00A0", " ");

                if (performer_name != "") {
                    performer_id = _dao.find_performer_by_exact_name(performer_name);
                    if (performer_id == null || performer_id <= 0) {
                        _dao.insert_performer(new Performer
                            (-1,
                             PerformerType.UNKNOWN,
                             performer_name
                            )
                        );
                        performer_id = _dao.find_performer_by_exact_name(performer_name);
                    }
                }
            }
        }
    }
}
```

```

    }

    if (raw.album != null) {
        string album_name = raw.album.strip().replace("\u00A0", " ");

        if (album_name != "" && album_name.downcase() != "unknown") {
            var existing_album = _dao.find_album_by_path(album_path);
            if (existing_album != null) {
                album_id = existing_album.get_id();
            } else {
                var new_album = new Album(-1, album_name, album_path, null);
                _dao.insert_album(new_album);
                album_id = new_album.get_id();
            }
        } else {
            album_id = null;
        }
    }

    var s = new Song(-1,
                    raw.title,
                    performer_id,
                    path,
                    album_id,
                    raw.genre,
                    raw.year,
                    raw.track
                    );
    _dao.insert_song(s);

    var s_obj = _dao.find_song_by_path(path);
    if (s_obj != null) {
        int s_id = s_obj.get_id();
    }

} catch (Error e) {
    stderr.printf("[ERROR] Failed to process file: %s : %s \n", path, e.message);
}
}
}

```

4.4. Modulo del controlador (GUI)

El módulo de la interfaz gráfica funciona como el controlador del sistema, siguiendo el patrón Modelo-Vista-Controlador (MVC). Este se encarga de gestionar la interacción del usuario con la aplicación, actuando como intermediario entre la vista (definida en los archivos de la interfaz) y el modelo (representado por el DAO, el Minero y las clases del dominio).

El controlador utiliza el objeto DAO para realizar operaciones sobre la base de datos, como consultas, inserciones y actualizaciones. Por ejemplo, al cargar una ventana de edición, se obtienen los datos correspondientes desde la base de datos y se muestran en la interfaz; posteriormente, cuando el usuario realiza modificaciones y guarda los cambios, el controlador actualiza la información mediante el DAO.

Asimismo, se manejan eventos mediante señales, permitiendo reaccionar a acciones del usuario como la creación o

edición de artistas, validación de formularios y guardado de datos. Esto permite mantener sincronizada la interfaz con el estado de la aplicación.

En general, todos los archivos del controlador siguen esta misma estructura: inicialización de la interfaz, carga de datos desde el modelo, manejo de eventos del usuario y actualización de la información en la base de datos. Veamos un ejemplo del constructor del dialogo para edición de canciones:

```
public EditSongDialog (Gtk.Window parent, Sonus.DAO.DAO dao, int id) {
    Object (transient_for: parent, modal: true);
    this.dao = dao;
    this.song_id = id;

    load_performers_dropdown();
    load_song_data(id);

    title_entry.notify["text"].connect(validate_forms);

    create_person_btn.clicked.connect (() => {
        var dialog = new PersonDialog(this, dao);
        dialog.set_transient_for(this);
        dialog.set_modal(true);
        dialog.present();
    });

    create_group_btn.clicked.connect (() => {
        var dialog = new GroupDialog(this, dao);
        dialog.set_transient_for(this);
        dialog.set_modal(true);
        dialog.present();
    });

    edit_artist_btn.clicked.connect (() => {
        edit_selected_artist();
    });

    cancel_btn.clicked.connect (() => {
        this.close();
    });

    save_btn.clicked.connect (() => {
        save_song(id);
    });
}
```

4.5. Módulo de consultas

Su implementación no fue completada por cuestiones de tiempo y organización, quedando como trabajo futuro.

4.6. Interfaz gráfica

La interfaz gráfica fue desarrollada utilizando GTK 4 junto con Blueprints, lo que permitió definir las vistas mediante archivos declarativos `.blp`. Estos archivos son compilados automáticamente a archivos `.ui`, los cuales son

utilizados por GTK en tiempo de ejecución. Esta separación facilita la mantenibilidad del código y sigue el principio de separación de responsabilidades.

Cada vista se define en un archivo `.blp`, donde se especifican los componentes gráficos (como ventanas, botones, entradas de texto y listas desplegables) y su disposición. Por ejemplo, en la ventana de edición de canciones se definen elementos como `Entry` para el título, `DropDown` para seleccionar el artista y botones para crear, editar o guardar cambios.

Estas vistas se conectan con clases en Vala mediante el uso de `GtkTemplate`, lo que permite acceder a los componentes definidos en la interfaz como atributos de la clase (por ejemplo, `title_entry`, `performers_dropdown`, etc.). De esta manera, el controlador puede leer y modificar directamente los valores mostrados en la interfaz.

Este diseño permite que la interfaz actúe como intermediaria entre el usuario y el modelo, manteniendo desacoplada la lógica de presentación de la lógica del modelo y facilitando futuras modificaciones o extensiones del sistema.

4.7. Pruebas unitarias

Se implementaron pruebas unitarias para todos los módulos desarrollados, con excepción de la interfaz gráfica, utilizando la biblioteca GLib. Estas pruebas permitieron verificar la correcta manipulación de datos, así como el comportamiento esperado de los componentes ante distintos escenarios.

En el módulo de dominio se realizaron pruebas exhaustivas sobre todas las entidades, validando tanto casos correctos como casos inválidos. Se verifican restricciones como campos vacíos, identificadores inválidos o rutas incorrectas, asegurando que no se puedan crear instancias con datos inconsistentes. Asimismo, se comprueba el correcto funcionamiento de los métodos *getter* y *setter*, garantizando que los objetos mantengan sus invariantes. Para aumentar la cobertura, se utilizaron datos generados de forma aleatoria.

En el módulo de base de datos se probaron tanto las operaciones del DAO como el comportamiento de la clase `DatabaseManager`. En particular, se validó la correcta implementación del patrón Singleton, la apertura y cierre de la conexión, así como la correcta creación del esquema de la base de datos.

Por otro lado, el DAO fue probado de forma integral, verificando operaciones como inserción, actualización y consultas (por id, ruta, género, artista, entre otras). Los resultados fueron contrastados tanto a nivel de objetos como directamente sobre la base de datos, asegurando la consistencia entre ambas representaciones.

En el módulo de minería de datos se validó el comportamiento de componentes como el lector de archivos, comprobando la correcta lectura de directorios, el filtrado por extensiones y el manejo adecuado de errores, como directorios inexistentes. Así como la correcta lectura de los metadatos.

Además, se implementaron mecanismos para aislar las pruebas, evitando efectos secundarios entre ejecuciones, lo que permitió garantizar resultados consistentes y reproducibles.

5. Elección de lenguaje(s) de programación

Se decidió utilizar Vala como lenguaje de programación principalmente por su integración nativa con GTK, lo cual facilitó el desarrollo de la interfaz gráfica del sistema. Además, Vala compila a C, lo que posibilita el uso directo de bibliotecas de bajo nivel como `taglib_c` para el manejo de metadatos en archivos MP3.

Asimismo, su sintaxis resulta familiar al ser similar a lenguajes como Java y C++, lo que redujo la curva de aprendizaje y permitió un desarrollo más ágil. Esta combinación de alto nivel en la escritura del código y bajo nivel en la ejecución lo convierte en una opción adecuada para aplicaciones de escritorio como la desarrollada en este proyecto.

6. Dificultades

A lo largo del desarrollo del proyecto se presentaron diversas dificultades tanto técnicas como de organización:

- **Lenguaje:** Se tuvieron dificultades iniciales con Vala debido a la falta de experiencia previa. En particular, los errores de compilación resultaron complejos de interpretar, ya que el código es traducido a C antes de compilarse. No obstante, el uso de herramientas como GDB permitió rastrear y comprender mejor los errores, facilitando su resolución.
- **Implementación guiada por pruebas:** La adopción de un enfoque basado en pruebas (TDD) representó un reto importante, ya que no se contaba con experiencia previa en este tipo de desarrollo. La escritura de pruebas exhaustivas requirió una inversión considerable de tiempo, lo que impactó directamente en el avance de otros módulos del sistema.
- **Gestión del tiempo:** Se dedicó una cantidad significativa de tiempo al diseño e implementación del DAO, priorizando la robustez y cobertura de pruebas. Sin embargo, esto afectó el progreso de otros componentes obligatorios, evidenciando la necesidad de una mejor planificación y distribución del esfuerzo.
- **Alcance del proyecto:** Debido a limitaciones de tiempo y organización, no fue posible implementar el módulo de consultas (buscador), quedando como trabajo futuro junto con el reproductor musical.

7. Resultados

El sistema desarrollado permite almacenar, consultar y modificar información de archivos MP3 mediante una base de datos. A través de la interfaz gráfica, el usuario puede visualizar canciones registradas, editar sus atributos (como título, género o artista) y gestionar la información asociada.

El módulo de minería permite recorrer directorios y extraer metadatos de archivos MP3, insertándolos automáticamente en la base de datos, evitando duplicados y asegurando la consistencia de la información. Por otro lado, el sistema implementa correctamente la capa de acceso a datos mediante el DAO, así como la validación de datos en el módulo de dominio, lo que garantiza la integridad de la información almacenada.

Sin embargo, algunas funcionalidades planteadas inicialmente no fueron completadas, como el módulo de consultas avanzadas y el reproductor de audio, quedando como trabajo futuro.

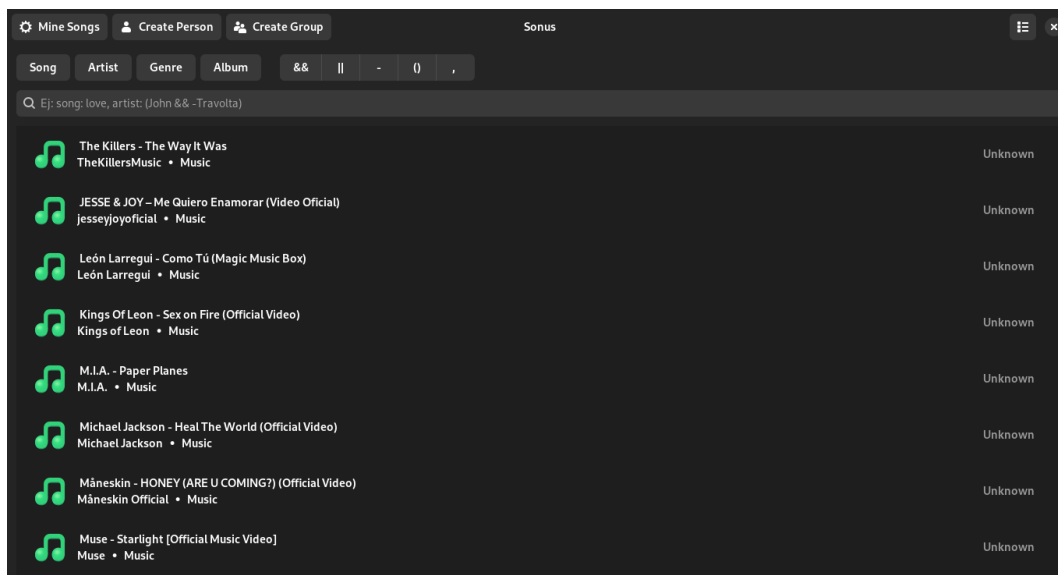


Figura 3: Interfaz del sistema Sonus

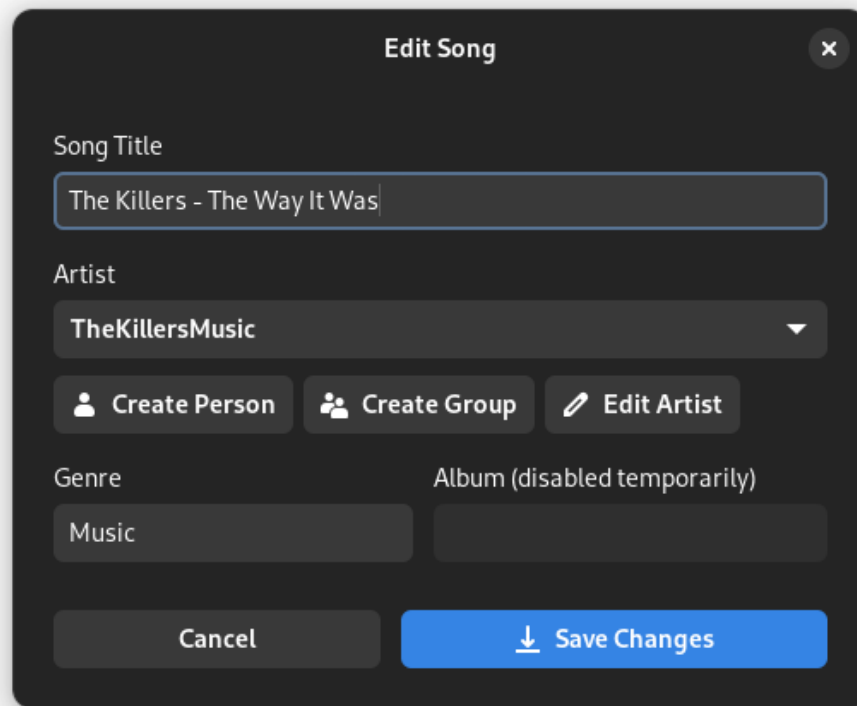


Figura 4: Dialogo para editar una canción

Los resultados obtenidos demuestran que la arquitectura modular implementada permite una correcta integración entre los distintos componentes del sistema. En particular, la interacción entre el módulo de minería y el DAO garantiza la inserción consistente de datos, mientras que la interfaz gráfica permite su manipulación de forma intuitiva.

Asimismo, las pruebas unitarias contribuyeron a validar el comportamiento del sistema, reduciendo errores y asegurando la estabilidad de los módulos principales.

8. Conclusiones

A lo largo del desarrollo de este proyecto se adquirieron diversos conocimientos técnicos, entre los que destacan el uso del lenguaje de programación Vala y sus bibliotecas asociadas, así como el manejo de metadatos en archivos MP3 mediante el estándar ID3v2.4. Asimismo, se aprendió a implementar pruebas unitarias y a desarrollar código guiado por pruebas, lo que permitió mejorar la calidad y confiabilidad del sistema. También se adquirió experiencia en el uso de herramientas de depuración como GDB, así como en el desarrollo de interfaces gráficas utilizando GTK y Blueprints, y en el manejo de bases de datos mediante SQLite.

Por otro lado, se identificaron áreas de mejora, particularmente en la gestión del tiempo y la organización del trabajo. En retrospectiva, se considera que una mejor distribución respecto a la importancia del detalle habría permitido dedicar menos tiempo al desarrollo de pruebas y más a la implementación de los módulos obligatorios del sistema.

Finalmente, como trabajo futuro, se plantea la implementación del módulo de consultas (buscador) y la inte-

gración de un reproductor de audio, con el objetivo de completar las funcionalidades originalmente planteadas.

En conclusión, el sistema desarrollado cumple en su mayoría con los objetivos planteados, proporcionando una base sólida sobre la cual se puede integrar la funcionalidad obligatoria faltante (buscador) así como la funcionalidad opcional faltante (reproductor).

9. Referencias

- Documentacion Vala:
 - Vala Documentation. (n.d.). <https://docs.vala.dev/>
 - Valadoc.org. (n.d.). <https://valadoc.org/>
- Documentación oficial de SQLite: SQLite Documentation. (n.d.). <https://sqlite.org/docs.html>
- Documentación de GTK: GTK-4.0. (n.d.). <https://docs.gtk.org/gtk4/>
- Documentación de Blueprint: Blueprint documentation. (n.d.). <https://blueprint-compiler-19b406.pages.gitlab.gnome.org/>
- Especificación ID3v2.4: ID3V2.4.0-structure - ID3.org. (n.d.). <https://id3.org/id3v2.4.0-structure>